Overview of Mixture of experts

Subject: Mixture of experts

1.

Routing In the original sparsely-gated MoE, only the top-k experts are queried, and their outputs are weighted-summed. There are other methods. Generally speaking, routing is an assignment problem: How to assign tokens to experts, such that a variety of constraints are followed (such as throughput, load balancing, etc.)? There are typically three classes of routing algorithm: the experts choose the tokens ("expert choice"), the tokens choose the experts (the original sparsely-gated MoE), and a global assigner matching experts and tokens. During inference, the MoE works over a large batch of tokens at any time. If the tokens were to choose the experts, then some experts might get few tokens, while a few experts get so many tokens that it exceeds their maximum batch size, so they would have to ignore some of the tokens. Similarly, if the experts were to choose the tokens, then some tokens might not be picked by any expert. This is the "token drop" problem. Dropping a token is not necessarily a serious problem, since in Transformers, due to residual connections, if a token is "dropped", it does not disappear. Instead, its vector representation simply passes through the feedforward layer without change. Other approaches include solving it as a constrained linear programming problem, using reinforcement learning to train the routing algorithm (since picking an expert is a discrete action, like in RL). The token-expert match may involve no learning ("static routing"): It can be done by a deterministic hash function or a random number generator.

2.

Meta-pi network The meta-pi network, reported by Hampshire and Waibel, uses f (x) = ∑ i w (x) i f i (x) {\displaystyle f(x)=\sum _{i}w(x)_{i}f_{i}(x)} as the output. The model is trained by performing gradient descent on the mean-squared error loss L := 1 N ∑ k ■ y k − f (x k) ■ 2 {\displaystyle L:={\frac {1}{N}}\sum _{k}\|y_{k}-f(x_{k})\|^{2}} . The experts may be arbitrary functions. In their original publication, they were solving the problem of classifying phonemes in speech signal from 6 different Japanese speakers, 2 females and 4 males. They trained 6 experts, each being a "time-delayed neural network" (essentially a multilayered convolution network over the mel spectrogram). They found that the resulting mixture of experts dedicated 5 experts for 5 of the speakers, but the 6th (male) speaker does not have a dedicated expert, instead his voice was classified by a linear combination of the experts for the other 3 male speakers.

3.

For each input-output pair (x , y) {\displaystyle (x,y)} , the weighting function is changed to increase the weight on all experts that performed above average, and decrease the weight on all experts that performed below average. This encourages the weighting function to learn to select only the experts that make the right predictions for each input. The i {\displaystyle i} -th expert is changed to make its prediction closer to y {\displaystyle y} , but the amount of change is proportional to w (x) i N (y | μ i , I) {\displaystyle w(x)_{i}N(y|\mu _{i},I)} . This has a Bayesian interpretation. Given input x {\displaystyle x} , the prior probability that expert i {\displaystyle i} is the right one is w (x) i {\displaystyle w(x)_{i}} , and N (y | μ i , I) {\displaystyle N(y|\mu _{i},I)} is the likelihood of evidence y {\displaystyle y} . So, w (x) i N (y | μ i , I) ∑ j w (x) j N (y | μ j , I) {\displaystyle {\frac

{w(x)_{i}N(y|\mu _{i},I)}{\sum _{j}w(x)_{j}N(y|\mu _{j},I)}}} is the posterior probability for expert i {\displaystyle i} , and so the rate of change for the i {\displaystyle i} -th expert is proportional to its posterior probability. In words, the experts that, in hindsight, seemed like the good experts to consult, are asked to learn on the example. The experts that, in hindsight, were not, are left alone. The combined effect is that the experts become specialized: Suppose two experts are both good at predicting a certain kind of input, but one is slightly better, then the weighting function would eventually learn to favor the better one. After that happens, the lesser expert is unable to obtain a high gradient signal, and becomes even worse at predicting such kind of input. Conversely, the lesser expert can become better at predicting other kinds of input, and increasingly pulled away into another region. This has a positive feedback effect, causing each expert to move apart from the rest and take care of a local region alone (thus the name "local experts").

4.

Variants The mixture of experts, being similar to the gaussian mixture model, can also be trained by the expectation-maximization algorithm, just like gaussian mixture models. Specifically, during the expectation step, the "burden" for explaining each data point is assigned over the experts, and during the maximization step, the experts are trained to improve the explanations they got a high burden for, while the gate is trained to improve its burden assignment. This can converge faster than gradient ascent on the log-likelihood. The choice of gating function is often softmax. Other than that, gating may use gaussian distributions and exponential families. Instead of performing a weighted sum of all the experts, in hard MoE, only the highest ranked expert is chosen. That is, f (x) = f arg ■ max i w i (x) (x) {\displaystyle f(x)=f_{\arg \max _{i}w_{i}(x)}(x)} . This can accelerate training and inference time. The experts can use more general forms of multivariant gaussian distributions. For example, proposed f i (y | x) = N (y | A i x + b i , Σ i) {\displaystyle f_{i}(y|x)=N(y|A_{i}x+b_{i},\Sigma _{i})} , where A i , b i , Σ i {\displaystyle A_{i},b_{i},\Sigma _{i}} are learnable parameters. In words, each expert learns to do linear regression, with a learnable uncertainty estimate. One can use different experts than gaussian distributions. For example, one can use Laplace distribution, or Student's t-distribution. For binary classification, it also proposed logistic regression experts, with f i (y | x) = { 1 1 + e β i T x + β i , 0 , y = 0 1 − 1 1 + e β i T x + β i , 0 , y = 1 {\displaystyle f_{i}(y|x)={\begin{cases}{\frac {1}{1+e^{\beta _{i}^{T}x+\beta _{i,0}}}},&y=0\\1-{\frac {1}{1+e^{\beta _{i}^{T}x+\beta _{i,0}}}},&y=1\end{cases}}} where β i , β i , 0 {\displaystyle \beta _{i},\beta _{i,0}} are learnable parameters. This is later generalized for multi-class classification, with multinomial logistic regression experts. One paper proposed mixture of softmaxes for autoregressive language modelling. Specifically, consider a language model that given a previous text c {\displaystyle c} , predicts the next word x {\displaystyle x} . The network encodes the text into a vector v c {\displaystyle v_{c}} , and predicts the probability distribution of the next word as S o f t m a x (v c W) {\displaystyle \mathrm {Softmax} (v_{c}W)} for an embedding matrix W {\displaystyle W} . In mixture of softmaxes, the model outputs multiple vectors v c , 1 , … , v c , n {\displaystyle v_{c,1},\dots ,v_{c,n}} , and predict the next word as ∑ i = 1 n p i S o f t m a x (v c , i W i) {\displaystyle \sum _{i=1}^{n}p_{i}\;\mathrm {Softmax} (v_{c,i}W_{i})} , where p i {\displaystyle p_{i}} is a probability distribution by a linear-softmax operation on the activations of the hidden neurons within the model. The original paper demonstrated its effectiveness for recurrent neural networks. This was later found to work for Transformers as well.